**USED MEDIAS LLC**
EMBEDDED SYSTEMS DIVISION

# Display Driver

UM-CYDOS-015

| DOCUMENT | REVISION | STATUS | CLASSIFICATION |
| --- | --- | --- | --- |
| UM-CYDOS-015 | 1.0 · 2026-08-14 | **Complete, verified on hardware** | Internal |

## 1. Abstract

cyd-os drives the CYD's ILI9341 panel: 240×320, 16-bit colour, filled regions and text, driven from a native task showing live kernel state.

There is **no framebuffer anywhere in the system**. UM-CYDOS-010 §7.2 argued that one should not exist — the panel holds the image in its own GRAM, so a host copy is a redundant 153,600 B against a 158,000 B heap. This report is that argument implemented: every pixel is rendered through a **480-byte** span buffer.

Confirmed on the physical panel: header bar, live numeric fields, an advancing marker, and a strip of colour primaries.

## 2. Pin map

Taken from the vendor project's active `TFT_eSPI/User_Setup.h` rather than from recollection. On a board where a wrong pin and a wrong init sequence produce an identical symptom, the difference between reading and remembering is the difference between a first-try success and an afternoon.

| Signal | GPIO | Note |
| --- | --- | --- |
| SCLK | 14 | |
| MOSI | 13 | |
| MISO | 12 | unused — the driver never reads the panel |
| CS | 15 | |
| DC | 2 | low = command, high = data |
| BL | 21 | active high |
| RST | — | **not wired to a GPIO**; follows the ESP32's own reset |

Because RST is not under software control, the only reset available after boot is the panel's `0x01 SWRESET`.

## 3. Why bit-banged SPI

Deliberate, and deliberately temporary.

Hardware SPI2 requires DPORT peripheral clock-enable bits and GPIO matrix signal routing. Every mistake in either produces **exactly the same symptom** as a wiring error, a wrong pin, or a bad init sequence: a black screen with no diagnostic. Bringing up four uncertain subsystems at once and then bisecting them from a single bit of output is how M2 consumed nine build cycles (UM-CYDOS-009 §6).

Bit-banging touches only GPIO registers, which `gpio.h` covers completely. A failure could therefore only be the panel, the pins, or the commands — three candidates instead of seven.

### 3.1 Measured cost

```
153,600 bytes in 387 ms  ≈ 397 kB/s  ≈ 3.2 MHz effective clock
```

**The estimate written into the source first was ~150 ms, wrong by a factor of 2.6.** A store to a peripheral register costs considerably more than the "few cycles" assumed. The figure is now measured at init and reported over UART, because throughput is the number that decides whether SPI2 is worth the risk, and an assertion would have decided it wrongly.

2.6 full screens per second is ample for a status display and useless for animation. Partial updates are what make it comfortable: redrawing six numeric fields costs a few hundred spans rather than 76,800 pixels.

## 4. Rendering without a framebuffer

`display_fill_rect()` composes one row into `g_line[240]` — 480 bytes — sets the panel's address window once, then pushes that span `h` times. `display_text()` does the same per glyph row.

The panel's GRAM is the framebuffer. Setting a window and streaming into it is not a workaround for having too little RAM; it is how the device is designed to be driven, and the 153,600 B figure in UM-CYDOS-010 §7 was only ever the cost of declining to use it.

### 4.1 The font is free

A 5×8 column-encoded font, 95 glyphs, 475 bytes, in `.rodata`. Since UM-CYDOS-011 that is mapped from flash through the data cache and costs **zero DRAM**. This is the first consumer of that work, and the reason it was scheduled before M4 rather than after.

### 4.2 Guarded by a mutex

The span buffer and the panel's current address window are both shared state. Two tasks drawing concurrently would interleave pixel streams into a single window and produce garbage.

`display.c` takes a recursive mutex around `fill_rect` and `text` — the first use of UM-CYDOS-014's primitive outside its own self-test. Recursive matters here: `display_clear()` draws through `display_fill_rect()`, so a non-recursive lock would deadlock on the first clear.

## 5. Verification

Visual confirmation on the panel: the status screen renders, values update, and the colour strip shows distinct primaries.

The colour strip exists as a diagnostic rather than decoration. Eight primaries in a known order make a pixel-format or byte-order fault immediately visible; a garbled or monochrome strip would have said which of the two was wrong, without inference from a photograph.

**Colour order confirmed.** Red renders leftmost, matching the intended `RED, GREEN, BLUE, YELLOW, CYAN, MAGENTA, WHITE, GREY`. That validates the BGR bit in `MADCTL 0x48`: had the panel's red and blue channels been transposed, the strip would have read blue-first and every colour in the system would have been mirrored while still looking entirely plausible in isolation. Distinct primaries alone would not have caught it — only their **order** does, which is why the strip is drawn in a known sequence rather than as arbitrary swatches.

The advancing marker block is the same idea applied to liveness: a display whose numbers all look plausible but never change is otherwise indistinguishable from a working one.

```
display : init ok, bytes=153634 fullscreen=387 ms
tasks   : report=0 a=1 b=2 vm=3 apps=4 shell=5 disp=6
```

`153,634` is 240×320×2 for the initial clear plus 34 command bytes — the exact figure a correct full-screen fill produces, which is what made it usable as a check before anything was known to be visible.

**Regression.** Eight native tasks now scheduling. M3, M4, M5 and locking self-tests all still passing.

## 6. Metrics

| Quantity | Value |
|---|---|
| Resolution | 240×320, RGB565 |
| Framebuffer | none |
| Span buffer | 480 B |
| Font | 475 B, in flash |
| Full-screen fill | 387 ms (measured) |
| Effective SPI clock | ~3.2 MHz |
| Bytes for init + clear | 153,634 |
| Native tasks | 8 |
| Image size | 18,896 B |

## 7. What this does not establish

- **No hardware SPI.** The 12× speedup from SPI2 at 40 MHz is unclaimed and unattempted. §3 explains why it was not attempted *first*; nothing prevents it now that the panel, pins and command sequence are known good.
- **No touch.** The CYD's XPT2046 is on separate pins and entirely untouched.
- **No orientation control.** MADCTL is fixed at `0x48`. Rotation is not implemented.

- **No clipping of text.** A string running past the right edge stops at the panel boundary rather than wrapping.
- **No damage tracking.** Callers decide what to redraw; nothing computes dirty regions.
- **No display access from applications.** There is no syscall for drawing, so bytecode cannot reach the screen. That is the obvious next step and would be the first VM syscall to carry a pointer, which needs the same bounds discipline as everything else in UM-CYDOS-012 §3.
- **No gamma correction.** The gamma tables ( `0xE0` / `0xE1` ) are left at panel defaults, so colours are correct in channel but not calibrated in response.

## 8. References

- UM-CYDOS-010 §7.2 — why no framebuffer, and the 153,600 B it would have cost
- UM-CYDOS-011 — flash-mapped `.rodata` , which the font depends on
- UM-CYDOS-014 — the mutex guarding the span buffer
- UM-CYDOS-009 §6 — the multi-variable bring-up this driver's staging avoids
- `kernel/gpio.h` — the two layers that must agree before a pin drives anything
- `kernel/display.c` — SPI, init sequence, span rendering, font